- *Register indirect addressing*. In this mode, the target memory address is taken from a register, rather than being encoded as a literal value in the operands of the instruction. This is how pointer dereferences are implemented in languages like C and C++. The value of the pointer (an address) is loaded into a register, and then a register-indirect "move" instruction is used to dereference the pointer and either load the value it points to into a target register, or store a value in a source register into that memory address.

- *Relative addressing*. In this mode, the target memory address is specified as an operand, and the value stored in a specified register is used as an offset from that target memory address. This is how indexed array accesses are implemented in a language like C or C++.

- *Other addressing modes*. There are numerous other variations and combinations, some of which are common to virtually all CPUs, and some of which are specific to certain CPUs.

3.4.7.5 Further Reading on Assembly Language

In the preceding sections, we've had just a tiny taste of assembly language. For an easy-to-digest description of x86 assembly programming, see http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html. For more on calling conventions and ABIs, see https://en.wikibooks.org/wiki/X86_Disassembly/Functions_and_Stack_Frames.

3.5 Memory Architectures

In a simple von Neumann computer architecture, memory is treated as a single homogeneous block, all of which is equally accessible to the CPU. In reality, a computer's memory is almost never architected in such a simplistic manner. For one thing, the CPU's registers are a form of memory, but registers are typically referenced by name in an assembly language program, rather than being addressed like ordinary ROM or RAM. Moreover, even "regular" memory is typically segregated into blocks with different characteristics and different purposes. This segregation is done for a variety of reasons, including cost management and optimization of overall system performance. In this section, we'll investigate some of the memory architectures commonly found in today's personal computers and gaming consoles, and explore some of the key reasons why they are architected the way they are.

3.5.1 Memory Mapping

An n-bit address bus gives the CPU access to a theoretical *address space* that is 2^n bytes in size. An individual memory device (ROM or RAM) is always addressed as a contiguous block of memory cells. So a computer's address space is typically divided into various contiguous segments. One or more of these segments correspond to ROM memory modules, and others correspond to RAM modules. For example on the Apple II, the 16-bit addresses in the range 0xC100 through 0xFFFF were assigned to ROM chips (containing the computer's firmware), while the addresses from 0x0000 through 0xBFFF were assigned to RAM. Whenever a physical memory device is assigned to a range of addresses in a computer's address space, we say that the address range has been *mapped* to the memory device.

Of course a computer may not have as much memory installed as could be theoretically addressed by its address bus. A 64-bit address bus can access 16 EiB of memory, so we'd be unlikely to ever fully populate such an address space! (At 160 TiB, even HP's prototype supercomputer called "The Machine" isn't anywhere close to that amount of physical memory.) Therefore, it's commonplace for some segments of a computer's address space to remain unassigned.

3.5.1.1 Memory-Mapped I/O

Address ranges needn't all map to memory devices—an address range might also be mapped to other *peripheral devices*, such as a joypad or a network interace card (NIC). This approach is called *memory-mapped I/O* because the CPU can perform I/O operations on a peripheral device by reading or writing to addresses, just as if they were oridinary RAM. Under the hood, special circuitry detects that the CPU is reading from or writing to a range of addresses that have been mapped to a non-memory device, and converts the read or write request into an I/O operation directed at the device in question. As a concrete example, the Apple II mapped I/O devices into the address range 0xC000 through 0xC0FF, allowing programs to do things like control bank-switched RAM, read and control voltages on the pins of a game controller socket on the motherboard, and perform other I/O operations by merely reading from and writing to the addresses in this range.

Alternatively, a CPU might communicate with non-memory devices via special registers known as *ports*. In this case, whenever the CPU requests that data be read from or written to a port register, the hardware converts the request into an I/O operation on the target device. This approach is called *port-mapped I/O*. In the Arduino line of microcontrollers, port-mapped I/O gives a

program direct control over the digital inputs and outputs at some of the pins of the chip.

3.5.1.2 Video RAM

Raster-based display devices typically read a hard-wired range of physical memory addresses in order to determine the brightness/color of each pixel on the screen. Likewise, early charcter-based displays would determine which character glyph to display at each location on the screen by reading ASCII codes from a block of memory. A range of memory addresses assigned for use by a video controller is known as *video RAM* (VRAM).

In early computers like the Apple II and early IBM PCs, video RAM used to correspond to memory chips on the motherboard, and memory addresses in VRAM could be read from and written to by the CPU in exactly the same way as any other memory location. This is also the case on game consoles like the PlayStation 4 and the Xbox One, where both the CPU and GPU share access to a single, large block of unified memory.

In personal computers, the GPU often lives on a separate circuit board, plugged into an expansion slot on the motherboard. Video RAM is typically located on the video card so that the GPU can access it as quickly as possible. A bus protocol such as PCI, AGP or PCI Express (PCIe) is used to transfer data back and forth between "main RAM" and VRAM, via the expansion slot's bus. This physical separation between main RAM and VRAM can be a significant performance bottleneck, and is one of the primary contributors to the complexity of rendering engines and graphics APIs like OpenGL and DirectX 11.

3.5.1.3 Case Study: The Apple II Memory Map

To illustrate the concepts of memory mapping, let's look at a simple real-world example. The Apple II had a 16-bit address bus, meaning that its address space was only 64 KiB in size. This address space was mapped to ROM, RAM, memory-mapped I/O devices and video RAM regions as follows:

```
0xC100 - 0xFFFF  ROM (Firmware)
0xC000 - 0xC0FF  Memory-Mapped I/O
0x6000 - 0xBFFF  General-purpose RAM
0x4000 - 0x5FFF  High-res video RAM (page 2)
0x2000 - 0x3FFF  High-res video RAM (page 1)
0x0C00 - 0x1FFF  General-purpose RAM
0x0800 - 0x0BFF  Text/lo-res video RAM (page 2)
0x0400 - 0x07FF  Text/lo-res video RAM (page 1)
```

```
0x0200 - 0x03FF  General-purpose and reserved RAM
0x0100 - 0x01FF  Program stack
0x0000 - 0x00FF  Zero page (mostly reserved for DOS)
```

We should note that the addresses in the Apple II memory map corresponded directly to memory chips on the motherboard. In today's operating systems, programs work in terms of *virtual* addresses rather than *physical* addresses. We'll explore virtual memory in the next section.

3.5.2 Virtual Memory

Most modern CPUs and operating systems support a memory remapping feature known as a *virtual memory system*. In these systems, the memory addresses used by a program don't map directly to the memory modules installed in the computer. Instead, whenever a program reads from or writes to an address, that address is first *remapped* by the CPU via a look-up table that's maintained by the OS. The remapped address might end up referring to an actual cell in memory (with a totally different numerical address). It might also end up referring to a block of data on-disk. Or it might turn out not to be mapped to any physical storage at all. In a virtual memory system, the addresses used by programs are called *virtual addresses*, and the bit patterns that are actually transmitted over the address bus by the memory controller in order to access a RAM or ROM module are called *physical addresses*.

Virtual memory is a powerful concept. It allows programs to make use of more memory than is actually installed in the computer, because data can overflow from physical RAM onto disk. Virtual memory also improves the stability and security of the operating system, because each program has its own private "view" of memory and is prevented from stomping on memory blocks that are owned by other programs or the operating system itself. We'll talk more about how the operating system manages the virtual memory spaces of running programs in Section 4.4.5.

3.5.2.1 Virtual Memory Pages

To understand how this remapping takes place, we need to imagine that the entire addressable memory space (that's 2^n byte-sized cells if the address bus is n bits wide) is conceptually divided into equally-sized contiguous chunks known as *pages*. Page sizes differ from OS to OS, but are always a power of two—a typical page size is 4 KiB or 8 KiB. Assuming a 4 KiB page size, a 32-bit address space would be divided up into 1,048,576 distinct pages, numbered from 0x0 to 0xFFFFF, as shown in Table 3.2.

From Address	To Address	Page Index
0x00000000	0x00000FFF	Page 0x0
0x00001000	0x00001FFF	Page 0x1
0x00002000	0x00002FFF	Page 0x2
...		
0x7FFF2000	0x7FFF2FFF	Page 0x7FFF2
0x7FFF3000	0x7FFF3FFF	Page 0x7FFF3
...		
0xFFFFE000	0xFFFFEFFF	Page 0xFFFFE
0xFFFFF000	0xFFFFFFFF	Page 0xFFFFF

Table 3.2. Division of a 32-bit address space into 4 KiB pages.

The mapping between virtual addresses and physical addresses is always done at the granularity of pages. One hypothetical mapping between virtual and physical addresses is illustrated in Figure 3.22.

3.5.2.2 Virtual to Physical Address Translation

Whenever the CPU detects a memory read or write operation, the address is split into two parts: the *page index* and an *offset* within that page (measured in bytes). For a page size of 4 KiB, the offset is just the lower 12 bits of the address, and the page index is the upper 20 bits, masked off and shifted to the right by 12 bits. For example, the virtual address 0x1A7C6310 corresponds to an offset of 0x310 and a page index of 0x1A7C6.

The page index is then looked up by the CPU's *memory management unit* (MMU) in a *page table* that maps virtual page indices to physical ones. (The page table is stored in RAM and is managed by the operating system.) If the page in question happens to be mapped to a physical memory page, the virtual page index is translated into the corresponding physical page index, the bits of this physical page index are shifted to the left as appropriate and ORed together with the bits of the original page offset, and voila! We have a physical address. So to continue our example, if virtual page 0x1A7C6 happens to map to physical page 0x73BB9, then the translated physical address would end up being 0x73BB9310. This is the address that would actually be transmitted over the address bus. Figure 3.23 illustrates the operation of the MMU.

If the page table indicates that a page is not mapped to physical RAM (either because it was never allocated, or because that page has since been

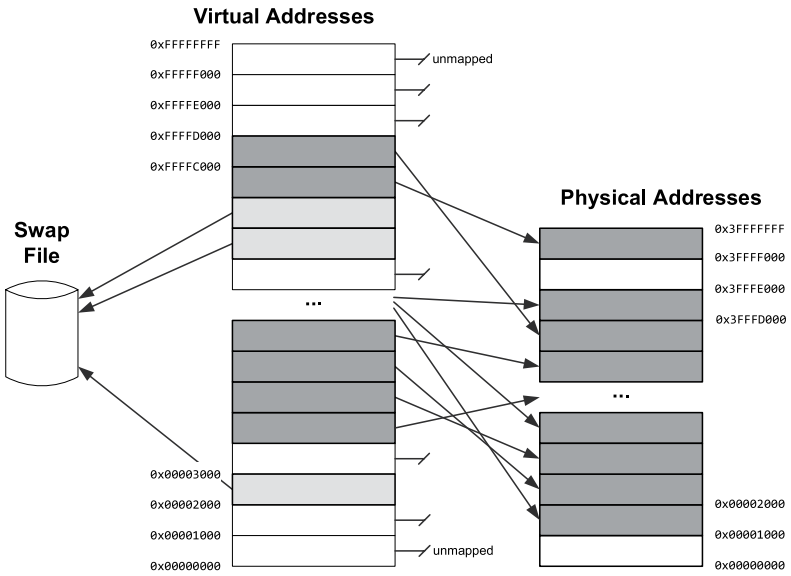

Figure 3.22. Page-sized address ranges in a virtual memory space are remapped either to physical memory pages, a swap file on disk, or they may remain unmapped.

swapped out to a disk file), the MMU raises an *interrupt*, which tells the operating system that the memory request can't be fulfilled. This is called a *page fault*. (See Section 4.4.2 for more on interrupts.)

3.5.2.3 Handling Page Faults

For accesses to unallocated pages, the OS normally responds to a page fault by crashing the program and generating a core dump. For accesses to pages that have been swapped out to disk, the OS temporarily suspends the currently-running program, reads the page from the swap file into a physical page of RAM, and then translates the virtual address to a physical address as usual. Finally it returns control back to the suspended program. From the point of view of the program, this operation is entirely seamless—it never "knows" whether a page was already in memory or had to be swapped in from disk.

Normally pages are swapped out to disk only when the load on the memory system is high, and physical pages are in short supply. The OS tries to swap out only the least-frequently-used pages of memory to avoid programs continually "thrashing" between memory-based and disk-based pages.

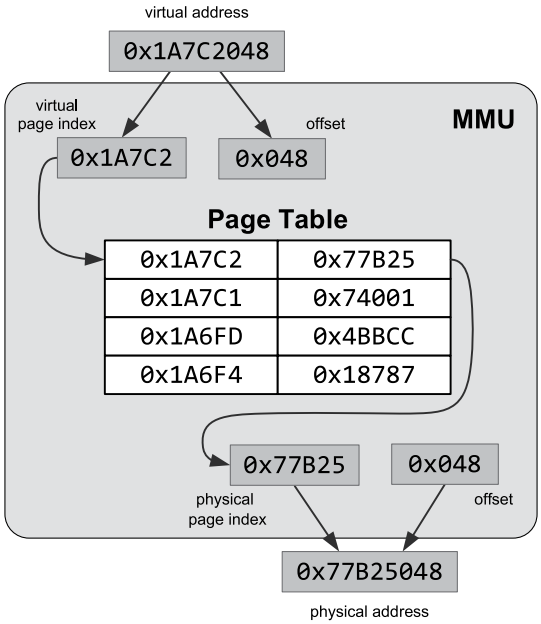

Figure 3.23. The MMU intercepts a memory read operation, and breaks the virtual address into a virtual page index and an offset. The virtual page index is converted to a physical page index via the page table, and the physical address is constructed from the physical page index and the original offset. Finally, the instruction is executed using the remapped physical address.

3.5.2.4 The Translation Lookaside Buffer (TLB)

Because page sizes tend to be small relative to the total size of addressable memory (typically 4 KiB or 8 KiB), the page table can become very large. Looking up physical addresses in the page table would be time-consuming if the entire table had to be scanned for every memory access a program makes.

To speed up access, a caching mechanism is used, based on the assumption that an average program will tend to reuse addresses within a relatively small number of pages, rather than read and write randomly across the entire address range. A small table known as the *translation lookaside buffer* (TLB) is maintained within the MMU on the CPU die, in which the virtual-to-physical address mappings of the most recently-used addresses are cached. Because this buffer is located in close proximity to the MMU, accessing it is very fast.

The TLB acts a lot like a general-purpose *memory cache hierarchy*, except that it is only used for caching page table entries. See Section 3.5.4 for a discussion of how cache hierarchies work.

3.5.2.5 Further Reading on Virtual Memory

See https://www.cs.umd.edu/class/sum2003/cmsc311/Notes/Memory/virtual.html and https://gabrieletolomei.wordpress.com/miscellanea/operating-systems/virtual-memory-paging-and-swapping for two good discussions of virtual memory implementation details.

This paper by Ulrich Drepper entitled, "What Every Programmer Should Know About Memory" is also a must-read for all programmers: https://www.akkadia.org/drepper/cpumemory.pdf.

3.5.3 Memory Architectures for Latency Reduction

The speed with which data can be accessed from a memory device is an important characteristic. We often speak of *memory access latency*, which is defined as the length of time between the moment the CPU requests data from the memory system and the moment that that data is actually received by the CPU. Memory access latency is primarily dependent on three factors:

1. the technology used to implement the individual memory cells,
2. the number of read and/or write ports supported by the memory, and
3. the physical distance between those memory cells and the CPU core that uses them.

The access latency of *static RAM* (SRAM) is generally much lower than that of *dynamic RAM* (DRAM). SRAM achieves its lower latency by utilizing a more complex design which requires more transistors per bit of memory than does DRAM. This in turn makes SRAM more expensive than DRAM, both in terms of financial cost per bit, and in terms of the amount of real estate per bit that it consumes on the die.

The simplest memory cell has a single *port*, meaing only one read or write operation can be performed by it at any given time. *Multi-ported* RAM allows multiple read and/or write operations to be performed simultaneously, thereby reducing the latency caused by contention when multiple cores, or multiple components within a single core, attempt to access a bank of memory simultaneously. As you'd expect, a multi-ported RAM requires more transistors per bit than a single-ported design, and hence it costs more and uses more real estate on the die than a single-ported memory.

The physical distance between the CPU and a bank of RAM also plays a role in determining the access latency of that memory. This is because electronic signals travel at a finite speed within the computer. In theory, electronic

signals are comprised of electromagnetic waves, and hence travel at close[7] to the speed of light. Additional latencies are introduced by the various switching and logic circuits that a memory access signal encounters on its journey through the system. As such, the closer a memory cell is to the CPU core that uses it, the lower its access latency tends to be.

3.5.3.1 The Memory Gap

In the early days of computing, memory access latency and instruction execution latency were on roughly equal footing. For example, on the Intel 8086, register-based instructions could execute in two to four cycles, and a main memory access also took roughly four cycles. However, over the past few decades both raw clock speeds and the effective instruction throughput of CPUs have been improving at a much faster rate than memory access speeds. Today, the access latency of main memory is extremely high relative to the latency of executing a single instruction: Whereas a register-based instruction still takes between one and 10 cycles to complete on an Intel Core i7, an access to main RAM can take on the order of 500 cycles to complete! This ever-increasing discrepancy between CPU speeds and memory access latencies is often called the *memory gap*. Figure 3.24 illustrates the trend of an ever-increasing memory gap over time.

Programmers and hardware designers have together developed a wide range of techniques to work around the problems caused by high memory access latency. These techniques usually focus on one or more of the following:

1. *reducing* average memory latency by placing smaller, faster memory banks closer to the CPU core, so that frequently-used data can be accessed more quickly;

2. "*hiding*" memory access latency by arranging for the CPU to do other useful work while waiting for a memory operation to complete; and/or

3. *minimizing* accesses to main memory by arranging a program's data in as efficient a manner as possible with respect to the work that needs to be done with that data.

In this section, we'll have a closer look at memory architectures for average latency reduction. We'll discuss the other two techniques (latency "hiding" and

[7] The speed that an electronic signal travels within a transmission medium such as copper wire or optical fiber is always somewhat lower than the speed of light in a vaccuum. Each interconnect material has its own characteristic *velocity factor* (VF) ranging from less than 50% to 99% the speed of light in a vaccuum.

the minimization of memory accesses via judicious data layout) while investigating parallel hardware design and concurrent programming techniques in Chapter 4.

3.5.3.2 Register Files

A CPU's register file is perhaps the most extreme example of a memory architecture designed to minimize access latency. Registers are typically implemented using multi-ported static RAM (SRAM), usually with dedicated ports for read and write operations, allowing these operations to occur in parallel rather than serially. What's more, the register file is typically located immediately adjacent to the circuitry for the ALU that uses it. Furthermore, registers are accessed pretty much directly by the ALU, whereas accesses to main RAM typically have to pass through the virtual address translation system, the memory cache hierarchy and cache coherence protocol (see Section 3.5.4), the address and data buses, and possibly also through crossbar switching logic. These facts explain why registers can be accessed so quickly, and also why the cost of register RAM is relatively high as compared to general-purpose RAM. This higher cost is justified because registers are by far the most frequently-used memory in any computer, and because the total size of the register file is very small when compared to the size of main RAM.

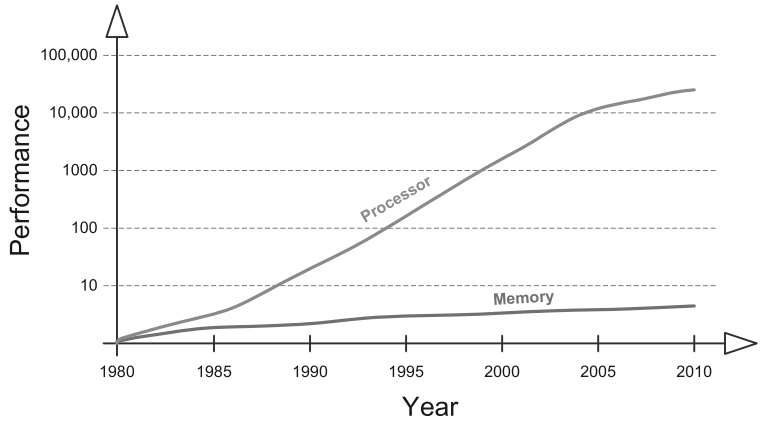

Figure 3.24. The ever-increasing difference between CPU performance and the performance of memory is called the *memory gap*. (Adapted from [23] *Computer Architecture: A Quantitative Approach* by John L. Hennessey and David A. Patterson.)

3.5.4 Memory Cache Hierarchies

A *memory cache hierarchy* is one of the primary mechanisms for mitigating the impacts of high memory access latencies in today's personal computers and game consoles. In a cache hierarchy, a small but fast bank of RAM called the *level 1* (L1) cache is placed very near to the CPU core (on the same die). Its access latency is almost as low as the CPU's register file, because it is so close to the CPU core. Some systems provide a larger but somewhat slower *level 2* (L2) cache, located further away from the core (usually also on-chip, and often shared between two or more cores on a multicore CPU). Some machines even have larger but more-distant L3 or L4 caches. These caches work in concert to automatically retain copies of the most frequently-used data, so that accesses to the very large but very slow bank of main RAM located on the system motherboard are kept to a minimum.

A caching system improves memory access performance by keeping *local copies* in the cache of those chunks of data that are most frequently accessed by the program. If the data requested by the CPU is already in the cache, it can be provided to the CPU very quickly—on the order of tens of cycles. This is called a *cache hit*. If the data is not already present in the cache, it must be fetched into the cache from main RAM. This is called a *cache miss*. Reading data from main RAM can take hundreds of cycles, so the cost of a cache miss is very high indeed.

3.5.4.1 Cache Lines

Memory caching takes advantage of the fact that memory access patterns in real software tend to exhibit two kinds of *locality of reference:*

1. *Spatial locality*. If memory address N is accessed by a program, it's likely that nearby addresses such as $N + 1$, $N + 2$ and so on will also be accessed. Iterating sequentially through the data stored in an array is an example of a memory access pattern with a high degree of spatial locality.

2. *Temporal locality*. If memory address N is accessed by a program, it's likely that that same address will be accessed again in the near future. Reading data from a variable or data structure, performing a transformation on it, and then writing an updated result into the same variable or data structure is an example of a memory access pattern with a high degree of temporal locality.

To take advantage of locality of reference, memory caching systems move data into the cache in contiguous blocks called *cache lines* rather than caching data items individually.

For example, let's say the program is accessing the data members of an instance of a class or struct. When the first member is read, it might take hundreds of cycles for the memory controller to reach out into main RAM to retrieve the data. However, the cache controller doesn't just read that one member—it actually reads a larger contiguous block of RAM into the cache. That way, subsequent reads of the other data members of the instance will likely result in low-cost cache hits.

3.5.4.2 Mapping Cache Lines to Main RAM Addresses

There is a simple one-to-many correspondence between memory addresses in the cache and memory addresses in main RAM. We can think of the address space of the cache as being "mapped" onto the main RAM address space in a repeating pattern, starting at address 0 in main RAM and continuing on up until all main RAM addresses have been "covered" by the cache.

As a concrete example, let's say that our cache is 32 KiB in size, and that cache lines are 128 bytes each. The cache can therefore hold 256 cache lines ($256 \times 128 = 32{,}768$ B $= 32$ KiB). Let's further assume that main RAM is 256 MiB in size. So main RAM is 8192 times as big as the cache, because $(256 \times 1024)/32 = 8192$. That means we need to overlay the address space of the cache onto the main RAM address space 8192 times in order to cover all possible physical memory locations. Or put another way, a single line in the cache maps to 8192 distinct line-sized chunks of main RAM.

Given any address in main RAM, we can find its address in the cache by taking the main RAM address *modulo* the size of the cache. So for a 32 KiB cache and 256 MiB of main RAM, the cache addresses 0x0000 through 0x7FFF (that's 32 KiB) map to main RAM addresses 0x0000 through 0x7FFF. But this range of cache addresses *also* maps to main RAM addresses 0x8000 through 0xFFFF, 0x10000 through 0x17FFF, 0x18000 through 0x1FFFF and so on, all the way up to the last main RAM block at addresses 0xFFF8000 through 0xFFFFFFF. Figure 3.25 illustrates the mapping between main RAM and cache RAM.

3.5.4.3 Addressing the Cache

Let's consider what happens when the CPU reads a single byte from memory. The address of the desired byte in main RAM is first converted into an address within the cache. The cache controller then checks whether or not the cache line containing that byte already exists in the cache. If it does, it's a cache hit, and the byte is read from the cache rather than from main RAM. If it doesn't, it's a cache miss, and the line-sized chunk of data is read from main RAM and

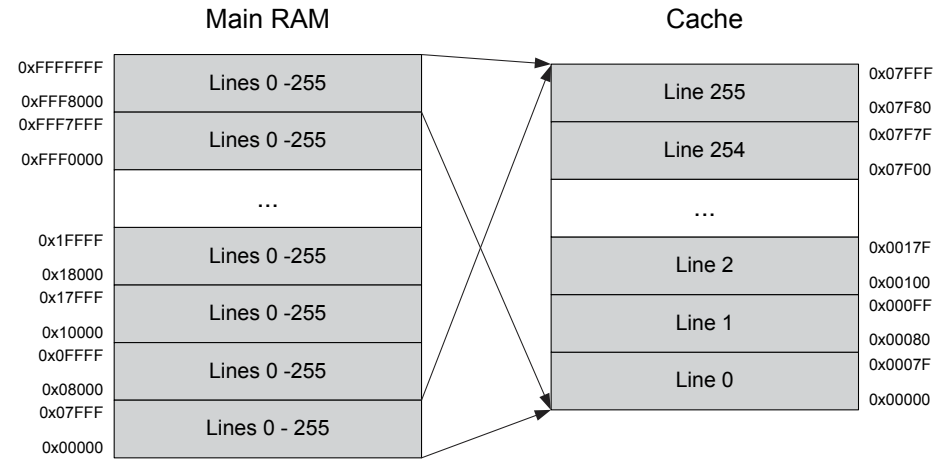

Figure 3.25. Direct mapping between main memory addresses and cache lines.

loaded into the cache so that subsequent reads of nearby addresses will be fast.

The cache can only deal with memory addresses that are *aligned* to a multiple of the cache line size (see Section 3.3.7.1 for a discussion of memory alignment). Put another way, the cache can really only be addressed in units of lines, not bytes. Hence we need to convert our byte's address into a *cache line index*.

Consider a cache that is 2^M bytes in total size, containing lines that are 2^n in size. The n least-significant bits of the main RAM address represent the *offset* of the byte within the cache line. We strip off these n least-significant bits to convert from units of bytes to units of lines (i.e., we divide the address by the cache line size, which is 2^n). Finally we split the resulting address into two pieces: The $(M - n)$ least-significant bits become the *cache line index*, and all the remaining bits tell us from *which* cache-sized block in main RAM the cache line came from. The block index is known as the *tag*.

In the case of a cache miss, the cache controller loads a line-sized chunk of data from main RAM into the corresponding line within the cache. The cache also maintains a small table of tags, one for each cache line. This allows the caching system to keep track of from *which* main RAM block each line in the cache came. This is necessary because of the many-to-one relationship between memory addresses in the cache and memory addresses in main RAM. Figure 3.26 illustrates how the cache associates a tag with each active line within it.

Returning to our example of reading a byte from main RAM, the complete

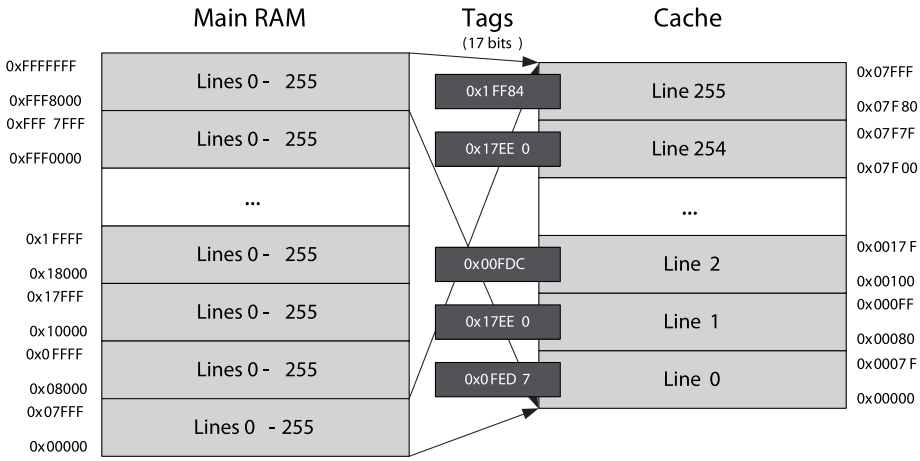

Figure 3.26. A *tag* is associated with each line in the cache, indicating from which cache-sized block of main RAM the corresponding line came.

sequence of events is as follows: The CPU issues a read operation. The main RAM address is converted into an offset, line index and tag. The corresponding tag in the cache is checked, using the line index to find it. If the tag in the cache matches the requested tag, it's a cache hit. In this case, the line index is used to retrieve the line-sized chunk of data from the cache, and the offset is used to locate the desired byte within the line. If the tags do not match, it's a cache miss. In this case, the appropriate line-sized chunk of main RAM is read into the cache, and the corresponding tag is stored in the cache's tag table. Subsequent reads of nearby addresses (those that reside within the same cache line) will therefore result in much faster cache hits.

3.5.4.4 Set Associativity and Replacement Policy

The simple mapping between cache lines and main RAM addresses described above is known as a *direct-mapped* cache. It means that each address in main RAM maps to only *one* line in the cache. Using our 32 KiB cache with 128-byte lines as an example, the main RAM address 0x203 maps to cache line 4 (because 0x203 is 515, and $\lfloor 515/128 \rfloor = 4$). However, in our example there are 8192 unique cache-line-sized blocks of main RAM that all map to cache line 4. Specifically, cache line 4 corresponds to main RAM addresses 0x200 through 0x27F, but also to addresses 0x8200 through 0x827F, and 0x10200 through 0x1027f, and 8189 *other* cache-line-sized address ranges as well.

When a cache miss occurs, the CPU must load the corresponding cache line from main memory into the cache. If the line in the cache contains no valid

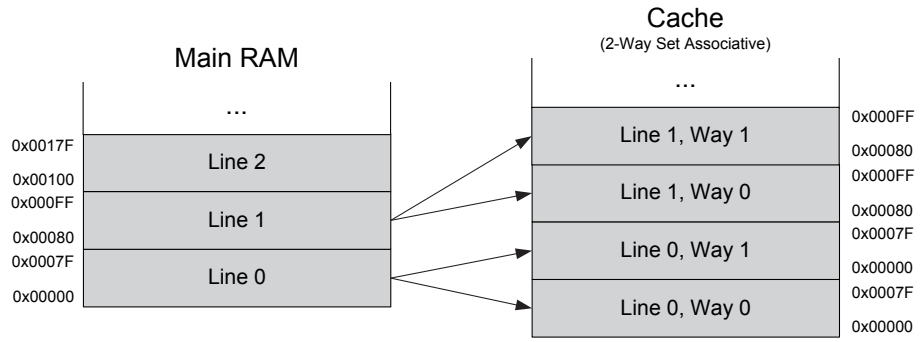

Figure 3.27. A 2-way set associative cache.

data, we simply copy the data into it and we're done. But if the line already contains data (from a different main memory block), we must overwrite it. This is known as *evicting* the cache line.

The problem with a direct-mapped cache is that it can result in pathological cases. For example, two unrelated main memory blocks might keep evicting one another in a ping-pong fashion. We can obtain better average performance if each main memory address can map to two or more distinct lines in the cache. In a *2-way set associative* cache, each main RAM address maps to two cache lines. This is illustrated in Figure 3.27. Obviously a 4-way set associative cache performs even better than a 2-way, and an 8-way or 16-way cache can outperform a 4-way cache and so on.

Once we have more than one "cache way," the cache controller is faced with a dilemma: When a cache miss occurs, which of the "ways" should we evict and which ones should we allow to stay resident in the cache? The answer to this question differs between CPU designs, and is known as the CPU's *replacement policy*. One popular policy is *not most-recently used* (NMRU). In this scheme, the most-recently used "way" is kept track of, and eviction always affects the "way" or "ways" that are *not* the most-recently used one. Other policies include *first in first out* (FIFO), which is the only option in a direct-mapped cache, least-recently used (LRU), least-frequently used (LFU), and pseudorandom. For more on cache replacement policies, see https://ece752.ece.wisc.edu/lect11-cache-replacement.pdf.

3.5.4.5 Multilevel Caches

The *hit rate* is a measure of how often a program hits the cache, as opposed to incurring the large cost of a cache miss. The higher the hit rate, the better the program will perform (all other things being equal). There is a fundamental

trade-off between cache latency and hit rate. The larger the cache, the higher the hit rate—but larger caches cannot be located as close to the CPU, so they tend to be slower than smaller ones.

Most game consoles employ at least two *levels* of cache. The CPU first tries to find the data it's looking for in the *level 1* (L1) cache. This cache is small but has a very low access latency. If the data isn't there, it tries the larger but higher-latency *level 2* (L2) cache. Only if the data cannot be found in the L2 cache do we incur the full cost of a main memory access. Because the latency of main RAM can be so high relative to the CPU's clock rate, some PCs even include *level 3* (L3) and *level 4* (L4) caches.

3.5.4.6 Instruction Cache and Data Cache

When writing high-performance code for a game engine or for any other performance-critical system, it is important to realize that both data and code are cached. The *instruction cache* (I-cache, often denoted I$) is used to preload executable machine code before it runs, while the *data cache* (D-cache, or D$) is used to speed up read and write operations performed by that machine code. In a level 1 (L1) cache, the two caches are always physically distinct, because it is undesirable for an instruction read to cause valid data to be bumped out of the cache or vice versa. So when optimizing our code, we must consider both D-cache and I-cache performance (although as we'll see, optimizing one tends to have a positive effect on the other). Higher-level caches (L2, L3, L4) typically do not make this distinction between code and data, because their larger size tends to mitigate the problems of code evicting data or data evicting code.

3.5.4.7 Write Policy

We haven't talked yet about what happens when the CPU *writes* data to RAM. How the cache controller handles writes is known as its *write policy*. The simplest kind of cache is called a *write-through cache*; in this relatively simple cache design, all writes to the cache are mirrored to main RAM immediately. In a *write-back* (or *copy-back*) cache design, data is first written into the cache and the cache line is only flushed out to main RAM under certain circumstances, such as when a dirty cache line needs to be evicted in order to read in a new cache line from main RAM, or when the program explicitly requests a flush to occur.

3.5.4.8 Cache Coherency: MESI, MOESI and MESIF

When multiple CPU cores share a single main memory store, things get more complicated. It's typical for each core to have its own L1 cache, but multiple cores might share an L2 cache, as well as sharing main RAM. See Figure 3.28

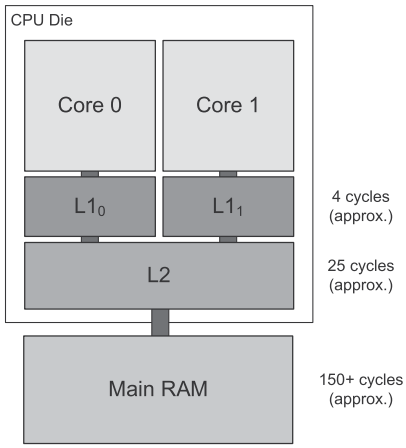

Figure 3.28. Level 1 and level 2 caches.

for an illustration of a two-level cache architecture with two CPU cores sharing one main memory store and an L2 cache.

In the presence of multiple cores, it's important for the system to maintain *cache coherency*. This amounts to making sure that the data in the caches belonging to multiple cores match one another and the contents of main RAM. Coherency doesn't have to be maintained at every moment—all that matters is that the running program can never *tell* that the contents of the caches are out of sync.

The three most common cache coherency protocols are known as MESI (modified, exclusive, shared, invalid), MOESI (modified, owned, exclusive, shared, invalid) and MESIF (modified, exclusive, shared, invalid and forward). We'll discuss the MESI protocol in more depth when we talk about multicore computing architectures in Section 4.9.4.2.

3.5.4.9 Avoiding Cache Misses

Obviously cache misses cannot be totally avoided, since data has to move to and from main RAM eventually. The trick to writing high performance software in the presence of a memory cache hierarchy is to arrange your data in RAM, and design your data manipulation algorithms, in such a way that the minimum number of cache misses occur.

The best way to avoid D-cache misses is to organize your data in *contiguous* blocks that are as *small* as possible and then access them *sequentially*. When the data is contiguous (i.e., you don't "jump around" in memory a lot), a single cache miss will load the maximum amount of relevant data in one go. When

the data is small, it is more likely to fit into a single cache line (or at least a minimum number of cache lines). And when you access your data sequentially, you avoid evicting and reloading cache lines multiple times.

Avoiding I-cache misses follows the same basic principle as avoiding D-cache misses. However, the implementation requires a different approach. The easiest thing to do is to keep your high-performance loops as small as possible in terms of code size, and avoid calling functions within your innermost loops. If you do opt to call functions, try to keep their code size small too. This helps to ensure that the entire body of the loop, including all called functions, will remain in the I-cache the entire time the loop is running.

Use inline functions judiciously. Inlining a small function can be a big performance boost. However, too much inlining bloats the size of the code, which can cause a performance-critical section of code to no longer fit within the cache.

3.5.5 Nonuniform Memory Access (NUMA)

When designing a multiprocessor game console or personal computer, system architects must choose between two fundamentally different memory architectures: *uniform memory access* (UMA) and *nonuniform memory access* (NUMA).

In a UMA design, the computer contains a single large bank of main RAM which is visible to all CPU cores in the system. The physical address space looks the same to each core, and each can read from and write to all memory locations in main RAM. A UMA architecture typically makes use of a cache hierarchy to mitigate memory access latency issues.

One problem with a UMA architecture is that the cores often contend for access to main RAM and any shared caches. For example, the PS4 contains eight cores arranged into two clusters. Each core has its own private L1 cache, but each cluster of four cores shares a single L2 cache, and all cores share main RAM. As such, the cores often contend with one another for access to the L2 cache and main RAM.

One way to address core contention problems is to employ a *non-uniform memory access* (NUMA) design. In a NUMA system, each core is provided with a relatively small bank of high-speed dedicated RAM called a *local store*. Like an L1 cache, a local store is typically located on the same die as the core itself, and is only accessible by that core. But unlike an L1 cache, access to the local store is explicit. A local store might be mapped to part of a core's address space, with main RAM mapped to a different range of addresses. Alternatively, certain cores might *only* be able to see the physical addresses within its local store, and might rely on a *direct memory access controller* (DMAC) to

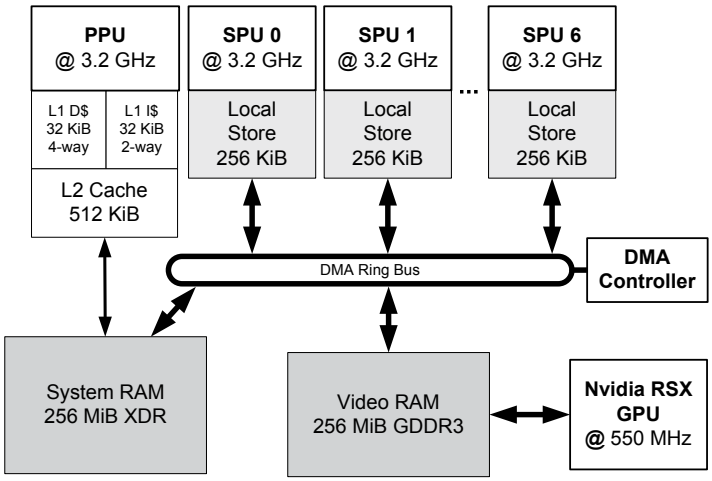

Figure 3.29. Simplified view of the PS3's cell broadband architecture.

transfer data between the local store and main RAM.

3.5.5.1 SPU Local Stores on the PS3

The PlayStation 3 is a classic example of a NUMA architecture. The PS3 contains a single main CPU known as the *Power processing unit* (PPU), eight[8] co-processors known as *synergistic processing units* (SPUs), and an NVIDIA RSX graphics processing unit (GPU). The PPU has exclusive access to 256 MiB of main system RAM (with an L1 and L2 cache), the GPU has exclusive access to 256 MiB of video RAM (VRAM), and each SPU has its own private 256 KiB local store.

The physical address spaces of main RAM, video RAM and the SPUs' local stores are all totally isolated from one another. This means that, for example, the PPU cannot directly address memory in VRAM or in any of the SPUs' local stores, and any given SPU cannot directly address main RAM, video RAM, or any of the other SPUs' local stores, only its own local store. The memory architecture of the PS3 is illustrated in Figure 3.29.

3.5.5.2 PS2 Scratchpad (SPR)

Looking even farther back to the PlayStation 2, we can learn about another memory architecture that was designed to improve overall system perfor-

[8]Only six of the SPUs are available for use by game applications—one SPU is reserved for use by the operating system, and one is entirely off-limits to account for inevitable flaws in the fabrication process.

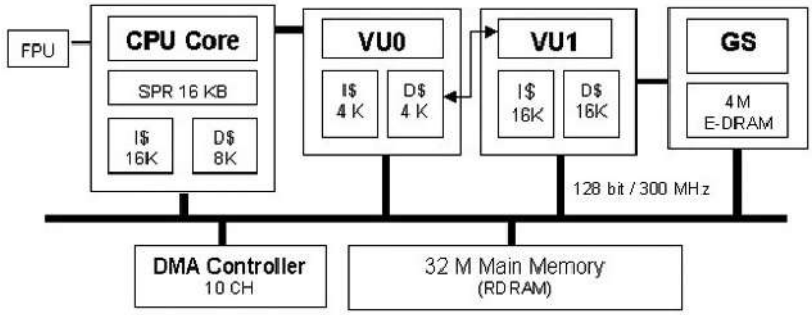

Figure 3.30. Simplified view of the PS2's memory architecture, illustrating the 16 KiB scratchpad (SPR), the L1 caches of the main CPU (EE) and the two vector units (VU0 and VU1), the 4 MiB bank of video RAM accessible to the graphics synthesizer (GS), the 32 MiB bank of main RAM, the DMA controller and the system buses.

mance. The main CPU on the PS2, called the emotion engine (EE), has a special 16 KiB area of memory called the *scratchpad* (abbreviated SPR), in addition to a 16 KiB L1 instruction cache (I-cache) and an 8 KiB L1 data cache (D-cache). The PS2 also contains two vector coprocessors known as VU0 and VU1, each with their own L1 I- and D-caches, and a GPU known as the *graphics synthesizer* (GS) connected to a 4 MiB bank of video RAM. The PS2's memory architecture is illustrated in Figure 3.30.

The scratchpad is located on the CPU die, and therefore enjoys the same low latency as L1 cache memory. But unlike the L1 cache, the scratchpad is memory-mapped so that it appears to the programmer to be a range of regular main RAM addresses. The scratchpad on the PS2 is itself uncached, meaning that reads and writes from and to it are direct; they bypass the EE's L1 cache entirely.

The scratchpad's main benefit isn't actually its low access latency—it's the fact that the CPU can access scratchpad memory without making use of the system buses. As such, reads and writes from and to the scratchpad can happen while the system's address and data bus are being used for other purposes. For example, a game might set up a chain of DMA requests to transfer data between main RAM and the two vector processing units (VUs) in the PS2. While these DMAs are being processed by the DMAC, and/or while the VUs are busy performing calculations (both of which would make heavy use of the system buses), the EE can be performing calculations on data that resides within the scratchpad, without interfering with the DMAs or the operation of the VUs. Moving data into and out of the scratchpad can be done via regular memory move instructions (or `memcpy()` in C/C++), but this task

can also be accomplished via DMA requests. The PS2's scratchpad therefore gives the programmer a lot of flexibility and power in order to maximize a game engine's data throughput.

4
Parallelism and Concurrent Programming

Computing performance—typically measured in millions of instructions per second (MIPS) or floating-point operations per second (FLOPS)—has been improving at a staggeringly rapid and consistent rate over the past four decades. In the late 1970s, the Intel 8087 floating-point coprocessor could muster only about 50 kFLOPS (5×10^4 FLOPS), while at roughly the same time a Cray-1 supercomputer the size of a large refrigerator could opeate at a rate of roughly 160 MFLOPS (1.6×10^8 FLOPS). Today, the CPU in game consoles like the Playstation 4 or the Xbox One produces roughly 100 GFLOPS (10^{11} FLOPS) of processing power, and the fastest supercomputer, currently China's Sunway TaihuLight, has a LINPACK benchmark score of 93 PFLOPS (peta-FLOPS, or a staggering 9.3×10^{16} floating-point operations per second). This is an improvement of seven orders of magnitude for personal computers, and eight orders of magnitude for supercomputers.

Many factors contributed to this rapid rate of improvement. In the early days, the move from vacuum tubes to solid-state transistors permitted the miniaturization of computing hardware. The number of transistors that could be etched onto a single chip rose dramatically as new kinds of transistors, new types of digital logic, new substrate materials and new manufacturing processes were developed. These advances also contributed to improvements in